

Návrhový vzor Proxy

s využitím cache

Cíl úkolu

Cílem tohoto cvičení je seznámit se s návrhovým vzorem **Proxy** v jazyce Java. Vaším úkolem bude navrhnout a implementovat jednoduchý systém, který bude využívat proxy k optimalizaci přístupu k časově náročným datům. Proxy třída bude poskytovat cachování (ukládání do mezipaměti) a zajistí, aby při opakovaných dotazech byly vráceny již načtené informace.

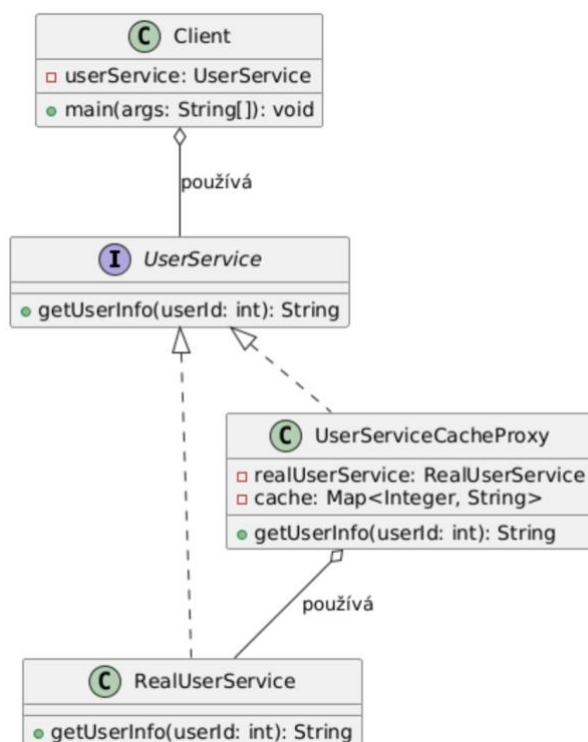
Popis systému

Vaše implementace bude obsahovat následující třídy:

1. **UserService** – rozhraní, které definuje metodu `getUserInfo`, jež vrátí informace o uživateli na základě jeho ID. Pro simulaci zpoždění použijte:

```
try {
    Thread.sleep(2000); // 2 sekundy zpoždění
} catch (InterruptedException e) {
    e.printStackTrace();
}
```
2. **RealUserService** – třída, která implementuje `UserService`. Simuluje časově náročné načítání dat (přístup do databáze) a vrátí informace o uživateli. Při každém volání metody `getUserInfo` dojde ke zpoždění, které napodobuje dlouhý čas načítání.
3. **UserServiceCacheProxy** – proxy třída, která také implementuje `UserService`. Tato třída obsahuje instanci `RealUserService` a zajišťuje, aby byla data načítána pouze jednou pro konkrétní uživatelské ID a následně uložena do cache.
4. **ProxyCachingExample** – hlavní třída, která bude sloužit jako testovací klient a bude využívat pouze metody `getUserInfo` přes proxy.

Diagram tříd (UML):



Požadavky na implementaci

1. **Implementujte jednotlivé třídy podle UML diagramu:**
 - Každá třída (UserService, RealUserService, UserServiceCacheProxy) by měla implementovat metody uvedené v diagramu, které simulují jejich funkce.
2. **Implementujte třídu UserServiceCacheProxy:**
 - Tato třída bude obsahovat instanci RealUserService a mezipaměť (cache) pro ukládání dat.
 - Při volání metody getUserInfo zkontrolujte, zda již dané ID existuje v cache:
 - Pokud ano, vraťte uložená data z cache.
 - Pokud ne, načtěte data z RealUserService, uložte je do cache a poté je vraťte.
3. **Implementujte funkci pro vyprázdnění cache:**
 - Do třídy UserServiceCacheProxy přidejte metodu clearCache, která vymaže všechna data uložená v cache.
4. **Vytvořte hlavní třídu ProxyCachingExample:**
 - Vytvořte instanci UserServiceCacheProxy.
 - Zavolejte metodu getUserInfo několikrát pro stejná i různá ID a sledujte, zda funguje cachování.
 - Vymažte cache pomocí metody clearCache a znovu otestujte přístup k datům.
 -

Specifikace metod

- **UserService**
 - getUserInfo(int userId): String – Vraťí informace o uživateli pro zadané ID.
- **RealUserService**
 - getUserInfo(int userId): String – Simuluje načítání dat z databáze. Před vrácením dat přidá zpoždění 2 sekundy (např. Thread.sleep(2000);).
- **UserServiceCacheProxy**
 - getUserInfo(int userId): String – Zkontroluje, zda jsou data pro zadané ID v cache. Pokud ano, vrátí je z cache, pokud ne, načte je z RealUserService a uloží do cache.
 - clearCache(): void – Vymaže všechna data z cache.

```
public class ProxyCachingExample {  
    public static void main(String[] args) {  
        UserService userService = new UserServiceCacheProxy();  
  
        System.out.println(userService.getUserInfo(1));  
        System.out.println(userService.getUserInfo(1));  
        System.out.println(userService.getUserInfo(2));  
        System.out.println(userService.getUserInfo(2));  
        System.out.println(userService.getUserInfo(1));  
    }  
}
```